

ReproLab

Polyglot Construction Roadmap

Express control plane + Go execution engine

24 weeks	1 runner	2 services	1 wow demo
Build window	MVP topology	Express + Go	Replay a real bug

Project thesis

Build a platform that turns a vague bug report into a shareable, executable reproduction. A user selects a repository and commit, runs a scripted browser scenario inside an isolated environment, and receives one synchronized timeline containing browser actions, network activity, application logs, screenshots, video and trace artifacts.

Prepared as an implementation guide, not an idea list.

1. Executive recommendation

Use a monorepo and build the smallest complete vertical slice first: public GitHub repository -> exact commit -> Docker build -> application startup -> Playwright scenario -> live logs -> artifact bundle -> rerun. Do not begin with private repositories, Kubernetes, database-level replay, automatic framework detection or arbitrary untrusted public code.

Important terminology

Your first release is reproducible rerunning, not deterministic program replay. ReproLab recreates the same source commit, environment recipe and browser actions. It cannot guarantee identical thread scheduling, third-party responses, random numbers or wall-clock behavior in arbitrary applications.

The recommended responsibility split

Component	Owns	Must not own
Express + TypeScript	Users, sessions, GitHub App integration, projects, scenarios, run records, authorization, quotas, signed artifact access and WebSocket fan-out.	Starting containers, executing repository commands or holding the Docker socket.
Go runner	Job leases, Git checkout, Docker build/run, health checks, timeout enforcement, capture orchestration, artifact upload, heartbeats and cleanup.	User-facing authorization decisions or long-lived GitHub credentials.
Playwright worker	Browser navigation, actions, assertions, screenshots, video, trace and normalized browser events.	Overall run state, container cleanup or user permissions.
PostgreSQL	Durable product metadata and runner lease state.	High-volume raw logs and videos.
Redis Streams	Durable-enough work delivery and low-latency live events.	Canonical long-term artifact storage.
S3-compatible storage	Immutable run bundles, trace files, videos, screenshots and compressed logs.	Run authorization logic.

Success criteria after six months

- **One-click reproduction:** A user connects a repository, selects a commit and launches a saved scenario.
- **Observable execution:** Build logs, service logs, browser actions, console messages and network requests appear in one live timeline.
- **Immutable evidence:** Every completed run produces a checksummed artifact bundle and a shareable read-only page.
- **Reliable rerun:** The same commit and environment definition can be launched again and compared against the earlier run.
- **Failure safety:** A killed or timed-out runner does not leave unlimited containers, networks or workspaces behind.
- **Credible isolation:** The production-style demo uses a dedicated runner host, strict limits and gVisor rather than executing code beside the control plane.

2. Define the product before the architecture

The narrow initial customer

The first customer is a web developer who can already express the bug as a Playwright scenario or a short action list. ReproLab is not initially a universal debugger. It is a reproducible web-application execution platform for Dockerized repositories.

The end-to-end user flow

1. Install the ReproLab GitHub App or paste a public repository URL.
2. Choose a branch or immutable commit SHA.
3. Select an environment recipe and a saved reproduction scenario.
4. Launch a run and watch build, startup and browser events live.
5. Open the synchronized timeline after the run finishes.
6. Share a read-only reproduction link with another developer.
7. Rerun the same inputs and compare the two executions.

The two-minute wow demonstration

- **0:00-0:20** - Open a deliberately broken shopping application and show a checkout action that sometimes fails.
- **0:20-0:40** - Open ReproLab, select the repository and the exact failing commit, then click Run.
- **0:40-1:15** - Show the live state transition from cloning to recording while service logs and browser events stream into the page.
- **1:15-1:40** - Open the failed request. The corresponding browser action, response status, backend error log and screenshot are aligned on one timeline.
- **1:40-2:00** - Click Rerun on a fixed commit and compare the successful response against the failed run.

The product is the synchronized evidence

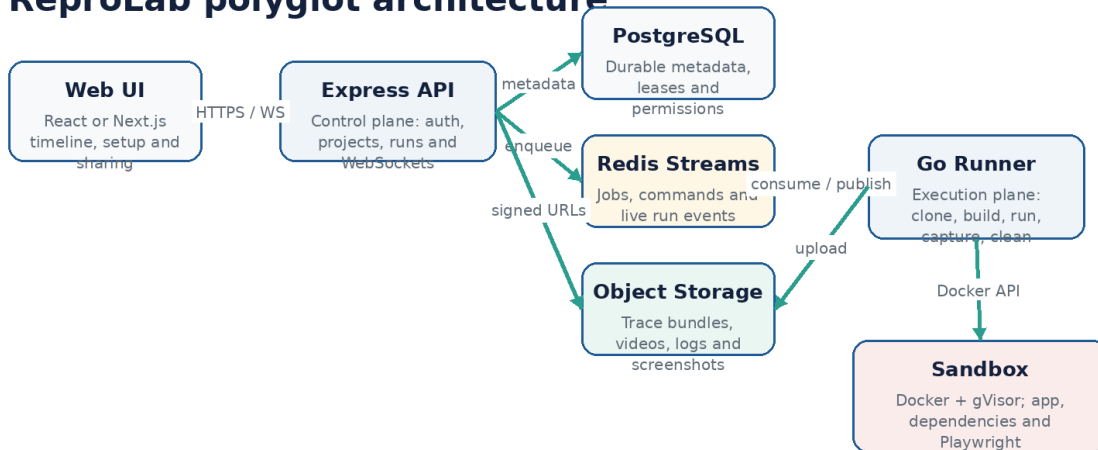
A collection of Docker commands plus a Playwright trace is not enough. The differentiating product is the normalized, searchable, shareable timeline that correlates what the user did with what the system emitted.

Strict MVP cut line

Build now	Delay
Public repositories	Private GitHub repositories
One Dockerized web service plus optional Postgres/Redis images	Arbitrary Docker Compose compatibility
Chromium only	Firefox and WebKit
Saved scenario DSL or committed Playwright script	Remote interactive browser recording
Container logs, browser console, requests, responses, screenshots, video and trace	Generic database change capture
Single Go runner	Autoscaled runner fleet
Trusted repositories only	Open public execution of arbitrary repositories
Rerun and basic diff	Deterministic instruction-level replay

3. System architecture

ReproLab polyglot architecture



The browser never talks directly to a runner. The Express API validates the request, records a pending run in PostgreSQL and appends a versioned job to Redis Streams. A Go runner claims the job, executes it, publishes live events and uploads large artifacts directly to object storage. Express exposes authorized metadata and signed download URLs.

Why asynchronous messaging is essential

- Builds and browser runs are long-running operations and should not occupy an HTTP request lifecycle.
- A runner may crash, disconnect or be restarted. A job requires a lease, acknowledgment and recovery path.
- Live events and durable metadata have different storage needs. Redis carries the live flow; PostgreSQL stores state; object storage stores the heavy evidence.
- The same contracts can later support multiple runner hosts without rewriting the API.

Redis Streams fit the first version

Streams provide an append-only event model and consumer groups with acknowledgments. Treat delivery as at-least-once: every job and event handler must be idempotent, and a runner must tolerate receiving a job again. [R8]

Recommended deployment boundaries

Environment	Control plane	Execution plane	Notes
Local development	Next.js, Express, Postgres, Redis and MinIO through Docker Compose.	Run the Go runner as a host process connected to a local rootless Docker Engine.	Avoid mounting a privileged Docker socket into the API container.
Portfolio deployment	Small API host or container service plus managed Postgres/Redis/object storage.	Dedicated Linux VM for the Go runner and sandbox runtime.	Do not place untrusted builds on the API host.
Later scale	Multiple stateless API replicas.	Runner pool grouped by capacity and runtime version.	Use leases, heartbeats and scheduling constraints before considering Kubernetes.

4. Express control plane

Primary modules

Module	Responsibilities
Auth	Session management, GitHub login, CSRF protection and account lifecycle.
GitHub	GitHub App installation records, repository listing, short-lived installation tokens and webhook verification.
Projects	Repository binding, allowed environments, team membership and default quotas.
Scenarios	Versioned action definitions or references to committed Playwright files.
Runs	Create, authorize, cancel, retry, compare and expose run metadata.
Artifacts	Manifest indexing and short-lived signed download URLs.
Realtime	Authorize WebSocket subscriptions and fan out normalized run events.
Runners	Runner registration view, heartbeats, capacity and administrative draining.

Control-plane rules

- A run is created transactionally before a job is published. Use an outbox table so a database commit cannot succeed while job publication is lost.
- Every state transition is compare-and-set. For example, only a QUEUED run can become LEASED and only the owning lease token can advance it.
- The API never accepts arbitrary shell commands. It accepts a validated, versioned ReproLab specification.
- GitHub installation access tokens are minted only when needed and passed to the assigned runner through the encrypted job payload or a one-time secret reference.
- Run pages use object-storage manifests rather than querying millions of log rows from PostgreSQL.

GitHub authentication design

Use a GitHub App rather than asking users for personal access tokens. Installation tokens can be scoped to repositories and permissions, can authenticate HTTP Git access, and expire after one hour. [R9]

Suggested REST surface

Initial API routes

```

POST /v1/projects
POST /v1/projects/:projectId/scenarios
POST /v1/projects/:projectId/runs
GET /v1/runs/:runId
POST /v1/runs/:runId/cancel
POST /v1/runs/:runId/rerun
GET /v1/runs/:runId/events?after=<sequence>
GET /v1/runs/:runId/artifacts
GET /v1/runs/:runId/compare/:otherRunId
WS /v1/runs/:runId/live

```

5. Go execution plane

Runner responsibilities

- **Lease a job:** Claim work with a unique attempt ID, heartbeat periodically and stop if the lease is revoked.
- **Prepare a workspace:** Create an isolated directory, fetch the exact commit and verify that the resolved SHA matches the job.
- **Validate the recipe:** Parse reprolab.yaml, apply platform limits and reject unsupported or dangerous options.
- **Build and launch:** Use the Docker Engine API from Go to build images, create an internal network and start declared services. [R10]
- **Wait for readiness:** Use bounded health checks with SSRF-safe destinations and explicit timeouts.
- **Record the scenario:** Launch a Playwright worker container on the same isolated network.
- **Collect evidence:** Stream logs and events, sample container metrics, assemble the artifact manifest and upload files.
- **Always clean:** Stop and remove containers, networks, temporary volumes and workspaces even after cancellation or failure.

Recommended Go package structure

```

services/runner/
  cmd/runner/          # process entrypoint and configuration
  internal/jobs/        # Redis Streams consumer and lease handling
  internal/contracts/   # versioned job and event types
  internal/gitx/        # clone, checkout and commit verification
  internal/spec/        # reprolab.yaml validation
  internal/executor/    # run state machine
  internal/dockerx/     # Engine API adapter
  internal/capture/     # logs, metrics and Playwright orchestration
  internal/artifacts/   # bundle manifest, compression and upload
  internal/security/    # limits, allowlists, redaction and policy
  internal/reaper/      # orphan discovery and cleanup
  internal/telemetry/   # runner traces, metrics and structured logs

```

Concurrency model

Use one goroutine per active run plus dedicated goroutines for job consumption, command consumption, heartbeat reporting and orphan reaping. Inside a run, use an errgroup-style cancellation tree: if cloning, building, startup or recording fails, cancel sibling collectors and move immediately to collection and cleanup. Bound total concurrency with a weighted semaphore based on runner capacity.

Do not confuse concurrency with capacity

A Go runner can manage many concurrent streams, but Docker builds and browsers consume real CPU, memory and disk. Start with one active run per runner host, then raise capacity only after collecting resource measurements.

6. Contracts between Express and Go

Keep the polyglot boundary explicit. Place JSON Schema or Protocol Buffer definitions in a shared contracts directory, version every message and generate TypeScript and Go types when practical. The runner should reject a job version it cannot understand rather than guessing.

Job envelope

Example run job

```
{
  "version": 1,
  "jobId": "job_01...",
  "runId": "run_01...",
  "attemptId": "attempt_01...",
  "repository": {
    "cloneUrl": "https://github.com/org/repo.git",
    "commitSha": "40-character-sha",
    "credentialRef": "secret://github/install-token/one-time-id"
  },
  "specObjectKey": "runs/run_01/spec.json",
  "limits": {
    "wallTimeSeconds": 900,
    "memoryBytes": 2147483648,
    "cpuCount": 2,
    "pids": 256,
    "diskBytes": 10737418240
  },
  "artifactUploadPrefix": "runs/run_01/attempt_01/"
}
```

Event envelope

Example normalized event

```
{
  "version": 1,
  "runId": "run_01...",
  "attemptId": "attempt_01...",
  "sequence": 184,
  "occurredAt": "2026-07-16T17:42:03.481Z",
  "source": "container:app",
  "type": "service.log",
  "payload": {
    "stream": "stderr",
    "message": "checkout failed: inventory reservation timeout"
  }
}
```

Delivery and idempotency rules

- A job is acknowledged only after its terminal status and artifact manifest are persisted.
- The unique key is (run_id, attempt_id). A duplicate delivery resumes or exits; it never creates a second independent attempt silently.
- Events have monotonically increasing runner-assigned sequence numbers within an attempt.
- Express deduplicates events by (attempt_id, sequence) before broadcasting or indexing.
- The canonical event stream is written as NDJSON into the artifact bundle. Redis may be trimmed after completion.
- Cancellation is a durable command, not only a volatile Pub/Sub message.

7. ReproLab environment specification

Avoid accepting arbitrary shell scripts from the web UI. The repository may contain normal build files, but the platform-facing configuration should be declarative and schema-validated. Start with a small vocabulary and expand it deliberately.

Example reprolab.yaml

```
version: 1

source:
  context: .
  dockerfile: Dockerfile
```

```

services:
  app:
    build: source
    internalPort: 3000
    health:
      path: /health
      timeoutSeconds: 60
    environment:
      NODE_ENV: test
      DATABASE_URL: secret://generated/postgres-url

  database:
    image: postgres:17-alpine
    environment:
      POSTGRES_DB: app
      POSTGRES_USER: app
      POSTGRES_PASSWORD: secret://generated/db-password

scenario:
  type: actions
  file: .reprolab/scenarios/checkout.yaml
  browser: chromium

capture:
  browserTrace: true
  video: true
  screenshots: on-failure
  console: true
  network: true
  websocketFrames: true
  serviceLogs: [app]

limits:
  wallTimeSeconds: 600
  memoryMiB: 2048
  cpus: 2
  pids: 256
  runtimeEgress: deny

```

Scenario action DSL

A limited action format is easier to validate and visualize than arbitrary JavaScript.

```

version: 1
steps:
  - goto: http://app:3000/login
  - fill: { selector: "[name=email]", value: "demo@example.com" }
  - fill: { selector: "[name=password]", secret: "demo-password" }
  - click: "button[type=submit]"
  - expectUrl: "**/checkout"
  - click: "[data-testid=place-order]"
  - expectResponse: { url: "**/api/orders", status: 201 }
  - screenshot: "after-order"

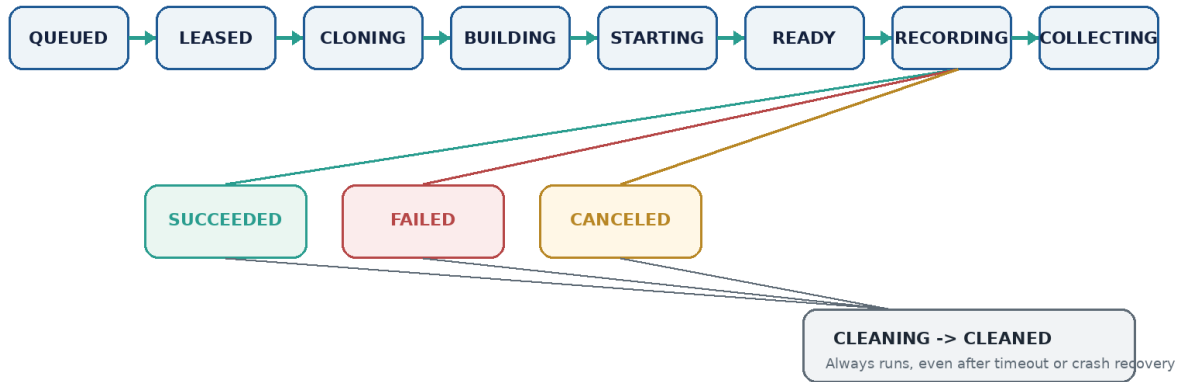
```

Support committed Playwright later

Advanced users will eventually need arbitrary Playwright tests. Add that mode after the action DSL works, and execute it only for trusted repositories. Keep the normalized event protocol identical in both modes.

8. Run lifecycle

Run lifecycle and failure-safe cleanup



State transition rules

State	Entry condition	Exit requirement
QUEUED	Run record and outbox item committed.	Runner claims the job and creates an attempt lease.
LEASED	Valid runner and lease token assigned.	Workspace created and exact commit resolved.
CLONING	Repository credentials available.	Checkout SHA verified and spec loaded.
BUILDING	Spec validated.	All required images built or pulled with digests recorded.
STARTING	Network and service definitions ready.	Required health checks pass.
READY	Application reachable from the Playwright worker.	Capture collectors started.
RECORDING	Browser scenario running.	Scenario passes, fails, times out or is canceled.
COLLECTING	Execution stopped.	Manifest and required artifacts uploaded.
TERMINAL	Success, failure or cancellation persisted.	Cleanup completes or is delegated to the reaper.

Crash recovery

- Each runner writes a heartbeat with its capacity and active attempt IDs.
- A lease expires if its heartbeat is not renewed within a short grace period.
- A recovery worker marks the attempt abandoned, appends a failure event and returns the job to the queue only when retry policy permits.
- Every Docker object receives labels for run ID, attempt ID and expiry time.
- The reaper scans for expired labeled containers, networks and volumes and removes them independently of the original goroutine.

9. Evidence capture and timeline design

Browser evidence

- Playwright trace archive for deep inspection in Trace Viewer. Trace Viewer can step through actions and inspect the recorded execution. [R1]
- Video recording, preferably retained on failure for cost control. Playwright supports configurable test video capture. [R2]
- Screenshots at named checkpoints and automatically on failure.
- Normalized action events: goto, click, fill, assertion start, assertion result and navigation.
- Console messages, uncaught page errors, request/response metadata and WebSocket frame metadata.

Service evidence

- Timestamped stdout and stderr from every selected container.
- Container start, stop, exit code, OOM status and health-check results.
- CPU, memory, process and network samples at a modest interval.
- Optional OpenTelemetry spans from applications that already emit OTLP. OpenTelemetry has current JavaScript and Go SDK documentation and supports traces, metrics and logs, but application logs should remain usable even without instrumentation. [R3] [R4]

Unified timeline rules

- Every event receives an attempt-local sequence number at ingestion.
- Store both event time and ingestion time. Never pretend distributed clocks provide perfect ordering.
- Use correlation IDs when available: trace ID, span ID, HTTP request ID and application-defined IDs.
- Group events into lanes: browser, network, app, database, infrastructure and annotations.
- A selected event should reveal surrounding evidence within a configurable time window.
- Redact secret-looking values before events leave the runner. Perform a second redaction pass before final bundle publication.

Artifact bundle

```
run-bundle/
  manifest.json
  source.json
  environment.json
  events.ndjson.gz
  checksums.sha256
  browser/
    trace.zip
    video.webm
    screenshots/
    network.har
  services/
    app.stdout.ndjson.gz
    app.stderr.ndjson.gz
    database.stdout.ndjson.gz
  metrics/
    containers.ndjson.gz
  annotations/
    comments.json
```

10. Rerun and comparison semantics

What must be pinned

- Exact Git commit SHA, not only a branch name.
- Built image digest or a rebuild record that identifies base-image digests.

- ReproLab specification version and scenario version.
- Generated dependency service image digests.
- Browser and Playwright image version.
- Declared environment variables after secret references are resolved, represented by safe fingerprints rather than plaintext.
- Runner runtime version and sandbox runtime version.

Comparison view

Dimension	Useful comparison
Scenario	First step where outcome diverged.
HTTP	Status, latency, headers and normalized body differences.
Console	New errors, removed errors and changed stack traces.
Service logs	First high-severity or correlated divergence.
Screenshots	Visual diff at matching named checkpoints.
Performance	CPU, memory and request-duration deltas.
Environment	Changed image digest, variable fingerprint or runtime version.

Determinism boundary

External APIs, time, randomness and concurrency can still produce different results. ReproLab should display nondeterminism indicators rather than claim certainty: unmocked external hosts, missing random seed, wall-clock access and changed dependency digests.

Later deterministic features

- HTTP fixture recording and replay for allowed external dependencies.
- Injected fixed clock and random seed for supported frameworks.
- Database snapshot restoration for a supported Postgres mode.
- Network delay and failure injection.
- Event-level comparison across repeated attempts to identify flaky steps.

11. Security model

Security reality

A normal Docker container is not a sufficient trust boundary for openly executing arbitrary hostile repositories. Begin with your own repositories and explicitly trusted demos. Move public multi-tenant execution behind a dedicated sandbox host and defense-in-depth isolation.

Minimum controls

Risk	Control
Host takeover	Dedicated runner VM; rootless Docker; gVisor runtime; no privileged containers; no host device access. Rootless mode runs the daemon and containers without root privileges, while gVisor

	integrates as an OCI runtime for sandboxed containers. [R6] [R7]
Resource exhaustion	Hard memory, CPU, PID, disk and wall-time limits. Docker containers have no resource limits by default, so every run must set them explicitly. [R5]
Docker socket abuse	Only the Go runner can reach the Engine API. The API and sandbox containers cannot access the socket.
Network abuse	Separate build and runtime networks; runtime egress denied by default; explicit hostname allowlist where necessary.
Secret theft	Short-lived credentials, secret references rather than values, in-memory delivery, redaction and no secret persistence in manifests.
Data leakage	Per-run object prefixes, encryption, signed short-lived URLs and authorization on every artifact request.
Orphaned resources	Labels, expiry timestamps, reaper process and startup reconciliation.
SSRF	Health checks only against declared internal services; disallow arbitrary metadata-service and host addresses.
Malicious archives	Limit checkout size, artifact count and compression ratios; reject unsafe paths.

Suggested container policy

```
runtime: runsc                # gVisor in the hardened demo
user: non-root
readOnlyRootFilesystem: true
capDrop: [ALL]
noNewPrivileges: true
memory: 2GiB
cpus: 2
pidsLimit: 256
runtimeNetwork: internal-only
timeout: 10m
hostMounts: none
privileged: false
```

Threat-model milestone

Before enabling repositories you do not control, write a one-page threat model covering assets, trust boundaries, attacker capabilities, abuse cases, residual risk and incident cleanup. This document is itself a strong portfolio artifact.

12. Durable data model

Table	Important fields
users	id, email, display_name, created_at
github_installations	id, user_id, installation_id, account_login, suspended_at
projects	id, owner_id, repository_full_name, default_branch, visibility
environment_specs	id, project_id, version, object_key, schema_version, checksum
scenarios	id, project_id, name, type, version, object_key, checksum
runs	id, project_id, requested_commit, resolved_commit, status, requested_by, created_at

run_attempts	id, run_id, runner_id, lease_token, lease_expires_at, status, started_at, ended_at
runner_nodes	id, name, runtime_version, capacity, last_heartbeat_at, draining
artifacts	id, attempt_id, kind, object_key, size_bytes, sha256, content_type
outbox	id, aggregate_type, aggregate_id, event_type, payload, published_at

What not to store in PostgreSQL

- Every log line from a long run.
- Video, screenshots, trace archives or compressed bundles.
- Raw GitHub installation tokens.
- Full unredacted environment variables.
- Large HTTP bodies by default.

Retention policy

- Keep failed run artifacts longer than successful run artifacts by default.
- Let project owners pin important runs.
- Delete expired object prefixes through a lifecycle rule and reconcile database artifact rows.
- Store small searchable summaries in PostgreSQL and the complete evidence in object storage.

13. Six-month construction roadmap

Each month ends with a demonstrable product increment. Do not spend a month building infrastructure that the UI cannot show. The order below continuously preserves a working vertical slice.

M1 Vertical slice	M2 Reliable sandbox	M3 Evidence timeline	M4 Rerun + compare
-----------------------------	-------------------------------	--------------------------------	------------------------------

M5 Security + GitHub	M6 Product polish	Weekly Demo checkpoint	Always Cut scope first
--------------------------------	-----------------------------	----------------------------------	----------------------------------

Month 1 - End-to-end skeleton

Week	Build	Acceptance test
1	Create monorepo, local infrastructure, sample broken web app and written MVP contract. Prototype the Go runner starting and removing a harmless container.	A single command starts Postgres, Redis, MinIO, Express, web UI and a host-process runner.
2	Implement projects, runs and attempts in Express. Add outbox publication to Redis	POST /runs creates a queued run; the runner claims it and updates visible state.

	Streams. Build the Go consumer and lease heartbeat.	
3	Clone a public repository, checkout an exact SHA, validate a minimal spec and build its Dockerfile.	The run page streams clone and build logs live.
4	Start the app container, wait for /health, capture stdout/stderr and clean all resources.	A deliberately broken app launches and its logs remain available after cleanup.

Month 1 deliverable

A user can launch a public repository at an exact commit and watch the full build/start lifecycle. No browser automation is required until this path is reliable.

Month 2 - Runner reliability and environment support

Week	Build	Acceptance test
5	Implement reprotlab.yaml schema, generated dependency services and an isolated per-run Docker network.	The sample app launches with Postgres using generated credentials.
6	Add CPU, memory, PID, disk and timeout policy; record image digests and exit/OOM data.	A memory-hog fixture is terminated and classified correctly.
7	Add cancellation commands, context propagation and guaranteed cleanup.	Cancel during build, startup and run; no labeled resources remain.
8	Implement runner startup reconciliation, orphan reaper and lease recovery.	Kill the runner process mid-run, restart it and verify orphan cleanup plus a coherent failure record.

Month 2 engineering targets

- One active run per runner host.
- All Docker objects labeled with run, attempt and expiry.
- No unbounded goroutines or channels.
- Every phase has an explicit timeout.
- At least five failure fixtures: clone failure, build failure, failed health check, timeout and OOM.

Month 3 - Browser recording and synchronized evidence

Week	Build	Acceptance test
9	Create Playwright worker image and action DSL. Run Chromium against the internal app hostname.	A saved scenario logs in and triggers the known bug without manual interaction.
10	Capture trace, video, screenshots, console and page errors.	The failed run opens in Playwright Trace Viewer and displays the expected screenshot.
11	Normalize request, response, action and WebSocket events; merge with service	Selecting the failed API request reveals the nearest backend error log.

	logs by sequence/time.	
12	Build the timeline UI with lanes, filtering, search and event details.	A reviewer can understand the failure without opening raw files.

Month 3 deliverable

This is the first true wow-factor release: a real bug executes inside an ephemeral environment and becomes a synchronized visual story.

Month 4 - Immutable artifacts, rerun and comparison

Week	Build	Acceptance test
13	Define manifest.json, NDJSON event format, compression, checksums and direct object-storage uploads.	A completed run can be reconstructed from its bundle without Redis.
14	Add signed artifact access, retention rules and read-only share links.	An unauthenticated share token can view only the authorized run and expires correctly.
15	Implement rerun from pinned inputs and environment fingerprinting.	A rerun resolves to the same commit and records any digest differences.
16	Implement side-by-side event, HTTP, screenshot and environment comparison.	A failed commit and fixed commit show the first meaningful divergence.

Comparison scope to keep manageable

- Compare named scenario steps, not arbitrary timeline alignment.
- Compare normalized JSON bodies only below a safe size threshold.
- Use image diff at matching screenshot names.
- Highlight changed environment fingerprints before suggesting a code regression.

Month 5 - Private repositories and hardened execution

Week	Build	Acceptance test
17	Register GitHub App, implement installation flow, repository selection and short-lived clone credentials.	A private test repository runs without storing a long-lived token.
18	Verify GitHub webhooks and optionally expose a commit status or pull-request check.	A configured PR can launch ReproLab and link to the result.
19	Move runner to a dedicated Linux VM; configure rootless Docker, gVisor, strict non-root policy and runtime egress restrictions.	A sandboxed run uses runsc and cannot reach blocked destinations or the Docker socket.
20	Add quotas, secret redaction tests, abuse limits and threat model.	Known secret fixtures never appear in UI events, bundles or logs.

Security milestone

Do not market this as safe arbitrary-code execution merely because gVisor is enabled. Document residual risk and keep the public demo allowlisted.

Month 6 - Product polish and portfolio launch

Week	Build	Acceptance test
21	Improve scenario authoring UI, validation errors and starter templates.	A new user can create a scenario without editing raw YAML.
22	Add annotations, share permissions, run naming and project activity view.	A reviewer can leave a comment attached to a specific timeline event.
23	Add metrics, structured telemetry, load tests and operational dashboards for the platform itself.	Ten queued fixture runs complete without duplicate attempts or leaked resources.
24	Finalize documentation, architecture decision records, security write-up, demo video, benchmark report and resume bullets.	A recruiter can run the demo from the README and understand the engineering tradeoffs.

Optional stretch features after week 24

- Remote interactive browser session and action recording.
- Postgres snapshot restore and selected-table before/after diff.
- External HTTP fixture recording and replay.
- Multi-run flakiness analysis.
- Runner autoscaling and scheduling by capacity or architecture.
- OpenTelemetry trace correlation inside the timeline.

14. The first 14 days in concrete tasks

Day	Output
1	Write README problem statement, MVP cut line and the two-minute demo script. Create the sample broken app.
2	Create monorepo folders, Docker Compose for Postgres/Redis/MinIO, linting and shared environment configuration.
3	Create runs and run_attempts migrations plus Express create/get run endpoints.
4	Create Redis Streams outbox publisher and a tiny Go consumer that prints the job.
5	Use the Docker Engine Go client to start, wait for and remove a hello-world fixture.
6	Publish runner status events and display them in a minimal browser run page.
7	Implement lease token, runner heartbeat and graceful shutdown.

8	Clone a public fixture repository into an ephemeral workspace and verify the commit SHA.
9	Parse a minimal reprolab.yaml and reject invalid paths, timeouts and unsupported fields.
10	Build the repository Dockerfile and stream build output.
11	Create an internal network, start the app and perform a bounded health check.
12	Stream app stdout/stderr and store a compressed log artifact in MinIO.
13	Add timeout/cancel flow and a defer-based cleanup stack plus labeled orphan reaper.
14	Record a screen demo of public repo -> run -> live logs -> artifact download. Write the next two-week issues from observed problems.

Day 14 checkpoint

Do not add Playwright until the build/start/log/cleanup path can survive repeated execution. A boring reliable runner is the foundation for the impressive parts.

15. Suggested monorepo layout

```

reprolab/
  apps/
    web/                # Next.js or React frontend
  services/
    api/                # Express + TypeScript control plane
    runner/             # Go execution engine
    playwright-worker/  # Browser capture image
  packages/
    contracts/          # schemas and generated TS/Go types
    ui/                 # optional shared UI components
  examples/
    broken-shop/        # primary demo fixture
    timeout-app/
    memory-hog/
  infra/
    docker-compose.dev.yml
    runner/
    migrations/
  docs/
    architecture/
    adr/
    threat-model.md
    artifact-format.md
  scripts/
    dev-up.sh
    seed-demo.sh
    cleanup-fixtures.sh

```

Repository discipline

- Use architecture decision records for major choices: Redis Streams, gVisor, artifact format and replay definition.
- Version contracts independently of service releases.
- Keep fixture applications intentionally small and deterministic enough to test failure paths.
- Do not share database models directly between TypeScript and Go. Share wire contracts, not implementation internals.

- Create one command that boots the complete development environment and seeds the demo project.

Branching and milestones

- One issue should produce one observable behavior or one safety property.
- Tag monthly demo releases: v0.1 vertical slice through v0.6 portfolio release.
- Keep a changelog with screenshots and short demo clips for each milestone.

16. Testing strategy

Layer	Examples
Express unit	Authorization, state transition guards, outbox creation, signed URL policy and GitHub webhook validation.
Go unit	Spec validation, lease behavior, cleanup stack, event sequencing, redaction and artifact manifest checksums.
Contract	The same fixture message validates against TypeScript and Go generated types.
Integration	Real Redis Streams, Postgres and MinIO; Docker Engine adapter tested with fixture containers.
End-to-end	Launch broken-shop, execute checkout scenario and assert timeline evidence plus artifacts.
Failure injection	Kill runner, expire lease, OOM container, fail health check, fill disk quota and interrupt upload.
Security	Secret fixtures, forbidden host mounts, blocked egress, path traversal and oversized artifact attempts.
Load	Many queued jobs, slow WebSocket clients and event-stream trimming.

The golden end-to-end fixture

Create a tiny application with a repeatable checkout bug: the frontend sends POST /api/orders, the backend emits a request ID, then fails when a seeded inventory row is absent. The expected ReproLab result should include the browser click, the 500 response, the backend log containing the same request ID, a failure screenshot and a non-zero scenario result.

Critical reliability tests

- Running the same job message twice does not create two uncontrolled attempts.
- A canceled run reaches a terminal state and removes all resources.
- A runner crash leaves resources that the reaper removes after expiry.
- Object upload failure leaves a visible partial-artifact error rather than reporting success.
- A user cannot subscribe to another project's run event stream.
- A secret included in application output is redacted before storage.

17. Operating ReproLab itself

Platform telemetry

- Run duration by phase: queue, clone, build, startup, record, collect and cleanup.
- Runner capacity, active attempts, lease expirations and orphan removals.
- Artifact upload bytes, failures and retention deletion counts.

- Redis pending entries and oldest unacknowledged job age.
- WebSocket subscribers and dropped/backpressured live events.
- Security counters: blocked egress, rejected specs, redaction matches and quota violations.

Internal correlation

Create one correlation context from the API request through the Redis job, Go runner logs and artifact manifest. Instrument Express and Go with OpenTelemetry for the platform itself before attempting to instrument every user application. [R3] [R4]

Operational runbooks

- Drain a runner without accepting new jobs.
- Recover a stuck pending Redis entry.
- Reconcile active attempts after a runner restart.
- Remove leaked resources by label and expiry.
- Rotate GitHub App keys and object-storage credentials.
- Disable public run creation during suspected abuse.

18. What not to build first

Temptation	Why it is dangerous now	Replacement
Kubernetes runner orchestration	Consumes weeks before one bug can be reproduced.	One dedicated VM plus sound leases and cleanup.
Automatic support for every language	Environment detection becomes the product instead of bug reproduction.	Require Dockerfile and a small platform spec.
Full Docker Compose compatibility	Compose semantics, volumes and networking explode scope.	Support a limited services section.
Generic database diff	Hard to secure and normalize across engines.	Postgres fixture reset; selected-table diff later.
Own trace viewer from day one	Rebuilds a mature debugging surface.	Store Playwright trace and build your cross-source timeline.
AI bug diagnosis	Distracts from high-quality evidence and becomes a wrapper.	Make evidence structured enough that diagnosis is obvious.
Billing and organizations	Adds CRUD without strengthening the technical demo.	Single-user or simple project membership.
Public arbitrary repo execution	Creates serious sandbox and abuse risk.	Trusted repositories and allowlisted demo users.

Scope rule

When behind schedule, remove a capture source or UI feature before weakening cleanup, state consistency or isolation. Reliability is part of the product.

19. Portfolio presentation plan

README structure

- Ten-second GIF of a failed run timeline.
- One-sentence problem and one-sentence solution.

- Architecture diagram with the Express/Go boundary.
- Why asynchronous jobs and at-least-once delivery were chosen.
- Security boundary and explicit trusted-repository limitation.
- Failure-recovery demonstration.
- Local quick start using the sample broken app.
- Benchmarks and known limitations.

Strong demo scenarios

- Backend exception correlated with a failed browser request.
- Flaky timeout reproduced under injected network delay.
- Memory leak terminated by limits and classified as OOM.
- Runner process killed mid-run, followed by lease expiry and resource cleanup.
- Failed commit compared against fixed commit.

Possible resume bullets after completion

Backend / infrastructure version

Built ReproLab, a polyglot bug-reproduction platform with an Express control plane and Go execution engine, orchestrating isolated Docker/gVisor environments through Redis Streams with leased jobs, live event streaming, hard resource limits and crash-safe cleanup.

Full-stack product version

Developed a shareable time-aligned debugging experience that correlates Playwright actions, HTTP traffic, console output, service logs, screenshots, video and trace artifacts; added exact-commit reruns and side-by-side failure comparison.

Security version

Designed defense-in-depth execution for trusted repositories using short-lived GitHub App credentials, dedicated runner hosts, rootless containers, gVisor, restricted egress, secret redaction and checksummed artifact bundles.

20. Definition of done

Area	Done when
Core flow	A repository and scenario produce a completed, shareable run without manual server intervention.
Correctness	State transitions are guarded and duplicate job delivery is safe.
Evidence	Browser, network and service evidence is synchronized and reconstructible from the bundle.
Rerun	The platform pins and displays all inputs needed to explain environmental differences.
Cleanup	Cancel, timeout, OOM, build failure and runner crash leave no permanent resources.
Security	The runner is separated from the API, credentials are short-lived, limits are mandatory and residual risk is documented.

Testing	The golden fixture and major failure paths run in CI or a repeatable integration environment.
Product	The two-minute demo is understandable to someone who has not read the code.
Documentation	Architecture, artifact format, threat model, local setup and known limitations are published.

Final prioritization

- **First:** Reliable run lifecycle and cleanup.
- **Second:** Browser scenario and synchronized timeline.
- **Third:** Immutable artifact bundle and rerun.
- **Fourth:** Private GitHub integration and hardened isolation.
- **Last:** Collaboration polish, advanced capture and scaling.

Best single milestone for a resume

If time becomes limited, finish through Month 4. A stable public-repository runner with Playwright evidence, artifact bundles and run comparison is already an unusually strong full-stack systems project.

Technical references

[R1] Playwright - Trace Viewer - <https://playwright.dev/docs/trace-viewer>

[R2] Playwright - Videos - <https://playwright.dev/docs/videos>

[R3] OpenTelemetry - Node.js getting started - <https://opentelemetry.io/docs/languages/js/getting-started/nodejs/>

[R4] OpenTelemetry - Go - <https://opentelemetry.io/docs/languages/go/>

[R5] Docker - Resource constraints - https://docs.docker.com/engine/containers/resource_constraints/

[R6] Docker - Rootless mode - <https://docs.docker.com/engine/security/rootless/>

[R7] gVisor - Docker Quick Start - https://gvisor.dev/docs/user_guide/quick_start/docker/

[R8] Redis - Streams and consumer groups - <https://redis.io/docs/latest/develop/use-cases/streaming/>

[R9] GitHub - Authenticating as a GitHub App installation - <https://docs.github.com/en/apps/creating-github-apps/authenticating-with-a-github-app/authenticating-as-a-github-app-installation>

[R10] Docker - Engine SDK examples - <https://docs.docker.com/reference/api/engine/sdk/examples/>

These references support the implementation choices in this roadmap. Versions and security guidance should be rechecked when the project is deployed because container runtimes, browser tooling and cloud platforms continue to evolve.